سد الفجوة بين XML و TypeScript (لسير عمل الترجمة)

رودولفو م. رايا rmraya@maxprograms.com **Maxprograms**
17 يونيو 2026
© Maxprograms 2026

يصف هذا المقال الانتقال من *Java* إلى *TypeScript* لمعالجة *XML* في سير عمل الترجمة، ومكتبات *TypesXML* و*TypesBCP47* و*TypesXLIFF* التي تم إنشاؤها لتوفير التحقق من صحة *DTD* و*XML Schema* لملفات *TMX* و*XLIFF* و*SRX* في *Node.js*.

مقدمة

إذا سبق لك أن حاولت معالجة XML في Node.js أو TypeScript، فمن المحتمل أنك واجهت قيدًا: فالتحليل سهل، لكن التحقق من الصحة وفقًا لـ DTD أو XML Schema ليس كذلك.

ويصبح ذلك مشكلة حقيقية عند العمل مع معايير الترجمة مثل TMX وXLIFF وSRX، حيث لا يُعتبر الهيكل والصحة أمرًا اختياريًا. فهما يحددان ما إذا كانت بياناتك قابلة للاستخدام أم لا.

هذه هي الفجوة التي واجهتها، وقررت في النهاية سدها.

الإعداد القديم: Java لـ XML

معايير الترجمة مثل TMX وXLIFF وSRX وTBX كلها قائمة على XML. عندما بدأت العمل بها منذ أكثر من 20 عامًا، كانت Java هي الخيار الأكثر عملية. لم تكن أدوات XML الخاصة بها مثالية، لكنها كانت موثوقة بما يكفي للتحليل والتحقق من الصحة.

وقد نجح ذلك لفترة طويلة.

لم تكن المشكلة في XML؛ بل في كل ما يحيط بها.

لم تتطور خيارات واجهة المستخدم لـ Java على سطح المكتب بطريقة تناسب احتياجاتي. استخدمت SWT لسنوات، لكن صيانتها أصبحت أقل، ولم تروق لي البدائل مثل Swing أو JavaFX.

الانتقال إلى TypeScript

حوالي عام 2018، تحولت إلى بناء واجهات المستخدم باستخدام HTML وCSS وJavaScript. في النهاية، أصبح TypeScript الخيار الواضح لأنه يوفر نموذج تطوير أكثر أمانًا مع فحوصات ثابتة.

وقد حل ذلك مشكلة واجهة المستخدم.

لكنه خلق مشكلة جديدة:

- واجهة المستخدم في TypeScript
- معالجة XML لا تزال تتم في Java

لم يكن الحفاظ على مجموعتين من التقنيات لسير عمل واحد أمراً مثالياً.

الجزء المفقود: التحقق من صحة XML في TypeScript

في ذلك الوقت، كانت هناك محللات XML في JavaScript، لكن لم تدعم أي منها:

- التحقق من صحة DTD
- التحقق من صحة مخطط XML

بالنسبة لتنسيقات الترجمة، يعد ذلك متطلبًا أساسيًا. فبدون التحقق من الصحة، لا يمكنك ضمان صحة بنية ملف TMX أو XLIFF.

لذلك بدأت في إنشاء **TypesXML**، وهو محلل ومدقق XML أصلي لـ TypeScript.

مثال بسيط

إليك كيف يبدو تحليل XML الأساسي:

```
import { Catalog, DOMBuilder, SAXParser } from "typesxml";

const parser: SAXParser = new SAXParser();
const handler: DOMBuilder = new DOMBuilder();
const catalog = new Catalog('/pathTo/catalog.xml');

parser.setCatalog(catalog);
parser.setContentHandler(handler);
parser.setValidating(true);
parser.parseFile("example.xml");

const document = handler.getDocument();
console.log(document.toString());
```

يمكن تكوين نفس المحلل من أجل:

- التحقق من صحة DTD
- التحقق من صحة XML Schema
- حل كتالوجات XML

وهذا يجعل من الممكن معالجة XML في Node.js بنفس مستوى التحكم المرتبط تقليديًا بمكتبات Java.

بناء المكدس المفقود

استغرق إنشاء مكدس XML كامل بعض الوقت:

- جاء تحليل DOM و SAX أولاً
- ثم تبع ذلك التحقق من صحة DTD
- كان التحقق من صحة مخطط XML هو الخطوة الأخيرة

يعد دعم مخطط XML أكثر تعقيدًا بشكل ملحوظ، ولكنه ضروري لتنسيقات مثل XLIFF.

العناصر الداعمة: بيانات اللغة

تعتمد سير عمل الترجمة أيضًا بشكل كبير على رموز اللغات.

يحدد BCP47 المعيار، لكن البيانات الرسمية تُوزَّع في شكل ملف نصي كبير. لتسهيل استخدامها في TypeScript، قمتُ بإنشاء **TypesBCP47**، وهي مكتبة مصاحبة تقوم بتحليل تلك البيانات وعرضها بطريقة منظمة.

إثبات المفهوم: إعادة كتابة الأدوات الحالية

للتحقق من صحة هذه الطريقة، قمت بإعادة كتابة الأدوات الحالية بلغة TypeScript:

- **SRXEditor**
- **TMXValidator**

تقوم هذه الأدوات بما يلي:

- تقوم بمعالجة ملفات XML كبيرة الحجم
- تتحقق من صحة البيانات وفقًا لقواعد النحو الرسمية
- تحافظ على أداء يضاهي الإصدارات الأصلية من Java

وقد أكد ذلك أن TypeScript قادرة على التعامل مع أحمال عمل الترجمة في العالم الواقعي.

دعم XLIFF

يستخدم XLIFF مخطط XML، مما تطلب تنفيذ أداة تحقق كاملة من صحة المخطط.

وبعد توفير ذلك، قمت بإنشاء **TypesXLIFF**، وهي مكتبة لتحليل ملفات XLIFF 2.x وإنشائها والتحقق من صحتها.

وهي تدعم:

- التحقق من صحة مخطط XML
- التحقق من الصحة وفقًا لمواصفات XLIFF

- نموذج كائنات مُصنف
- تحويل JSON

ما يتيحه ذلك

بعد توفر هذه العناصر، أصبح من الممكن الآن:

- التحقق من صحة ملفات TMX وXLIFF وSRX مباشرةً في Node.js
- إنشاء مسارات عمل الترجمة بالكامل في TypeScript
- التخلص من الحاجة إلى خادم خلفي منفصل يعمل بلغة Java
- الحفاظ على كل من واجهة المستخدم والمعالجة بنفس اللغة

ما هي الخطوة التالية

يجري حالياً العمل على دعم TBX، إلى جانب تحديثات لأدوات أخرى لنقلها بالكامل إلى TypeScript.

لا يزال بعض كود Java موجودًا في المكونات القديمة، لكن التطوير الجديد يركز الآن على TypeScript.

خاتمة

لفترة طويلة، كانت معالجة XML في سير عمل الترجمة تتطلب فعليًا استخدام Java.

لم يعد الأمر كذلك.

إذا كنت تعمل بتنسيقات تستند إلى XML في Node.js، خاصة في الترجمة أو مسارات البيانات المنظمة، فيمكنك الآن التعامل مع التحليل والتحقق من الصحة والمعالجة بالكامل باستخدام TypeScript.

نبذة عن المؤلف



يشارك رودولفو م. رايا، المدير التقني لشركة Maxprograms SAS، في تطوير معايير الترجمة، بما في ذلك XLIFF وTMX وTBX وSRX وGlossML.

وهو يطور الأدوات والمكتبات التي تنفذ هذه المواصفات، بما في ذلك البرامج وخدمات التحقق من الصحة المتوفرة على هذا الموقع.